



Robotics Interest Group



ROUND 1 INDUCTION REPORT

Name:	Adithya A
Roll No:	B250611EC
Phone No:	9895266671
Link to Google drive/project:	All the videos and links are there in this notion document: <u>Robot-Radar-Adithya-A-B250611EC</u>
Remarks:	Engineered an ESP32-powered intelligent robotic scanning system that integrates autonomous detection, validated control logic, and real-time web visualization into a cohesive, interactive embedded platform.

Problem Statement:

Design and simulate a robot head using an HC-SR04 ultrasonic sensor, micro servo, and piezo buzzer. The servo should sweep between 0° and 180° until an object is detected within 20cm, at which point it freezes and the buzzer beeps rapidly.

Solution:

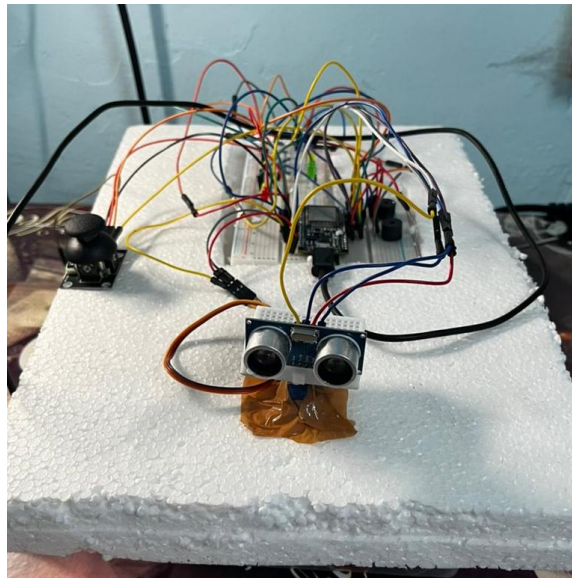
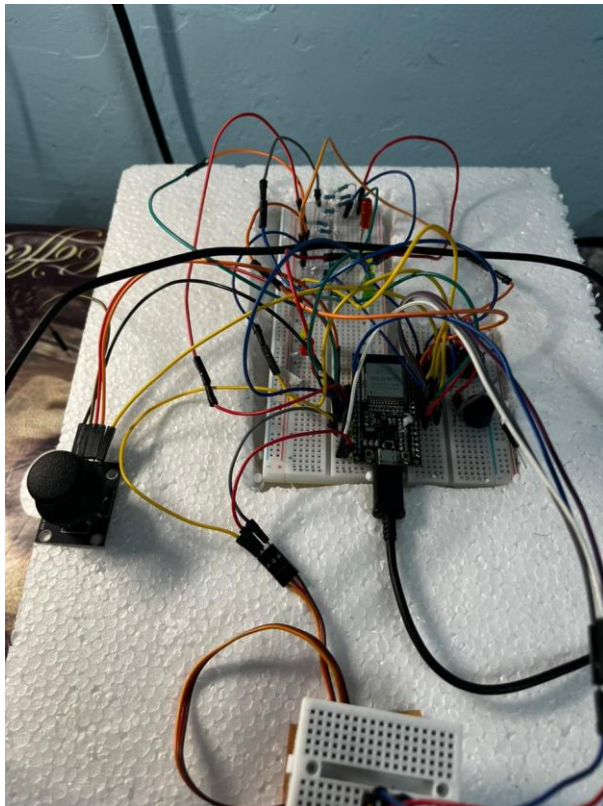
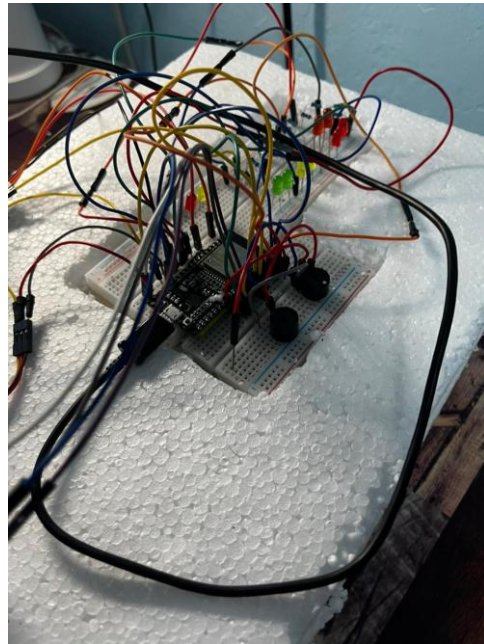
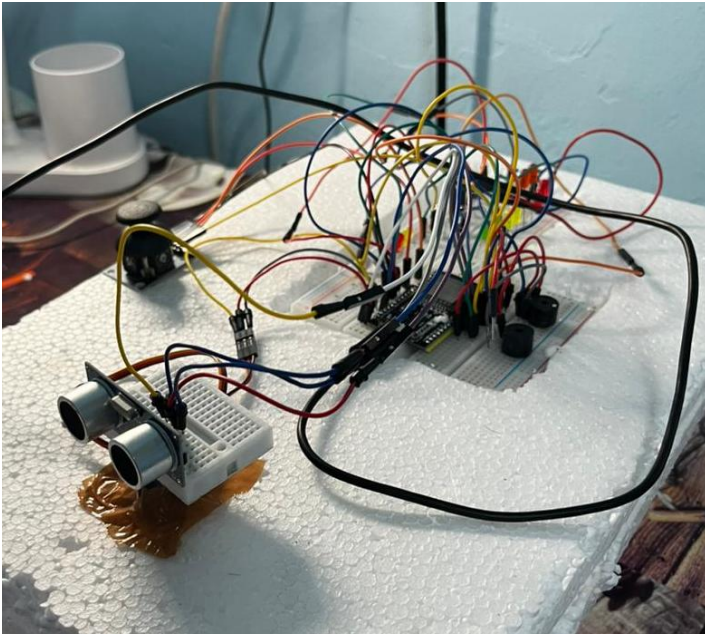
I built an intelligent ESP32-based robotic scanning system that actively monitors its environment and reacts in real time using an HC-SR04 ultrasonic sensor and a micro servo. The system autonomously sweeps between 0° and 180°, and when an object enters a defined 20 cm detection range, it validates the signal, locks onto the target, and provides graded visual feedback through a proximity LED ladder along with parking-sensor-style audio alerts.

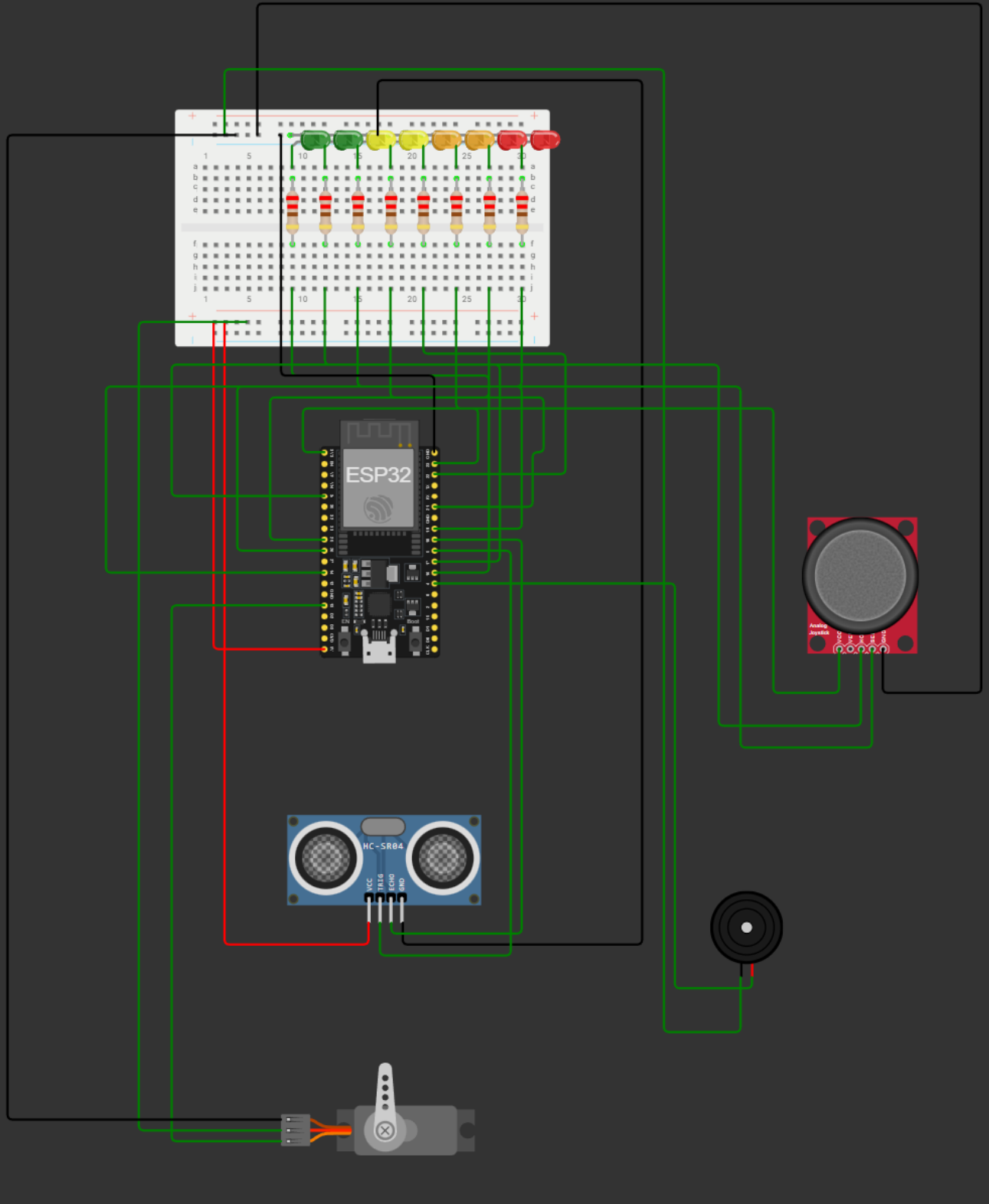
The robot operates in two modes: in **Auto mode**, it independently scans and freezes upon confirmed object detection, while in **Manual mode**, the servo can be precisely controlled via a joystick for interactive operation and testing.

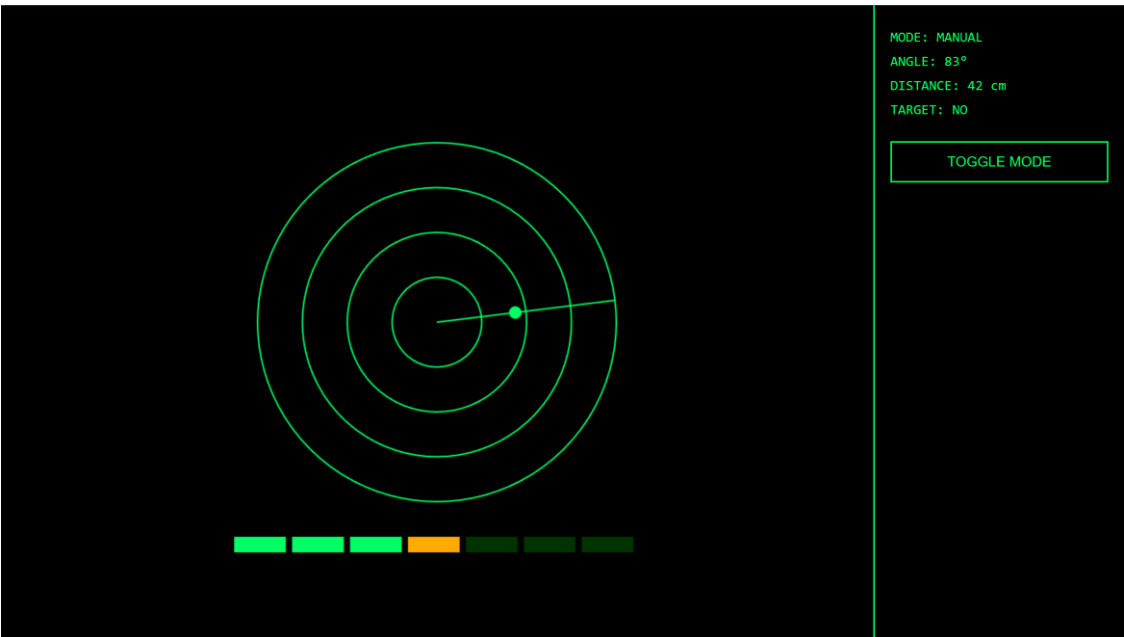
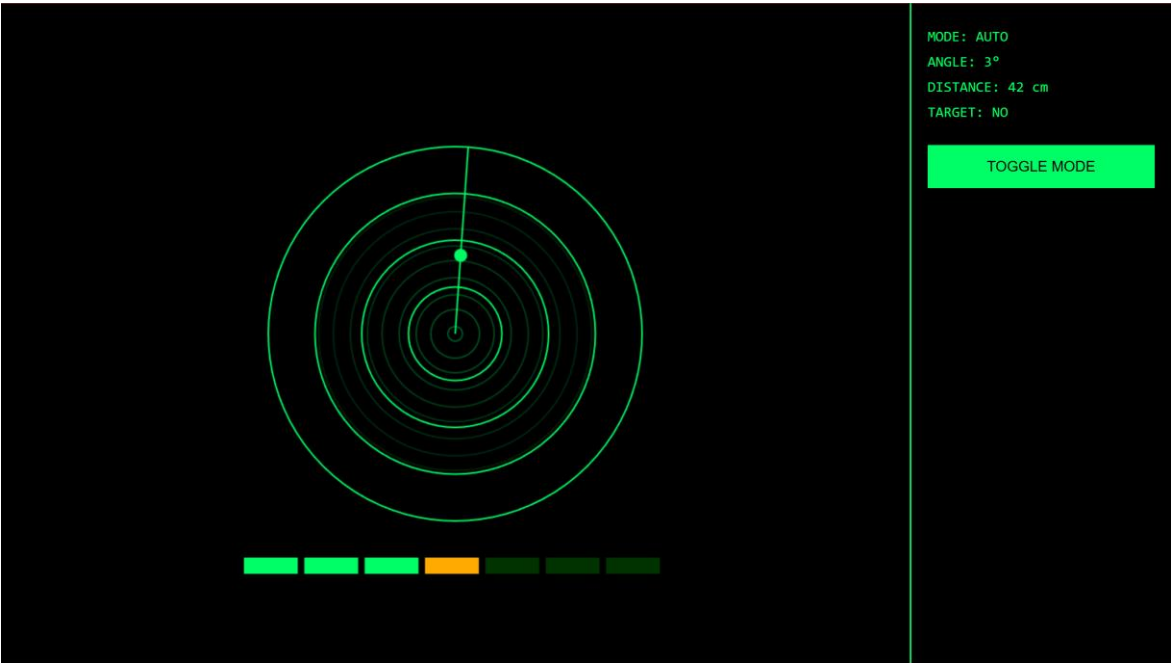
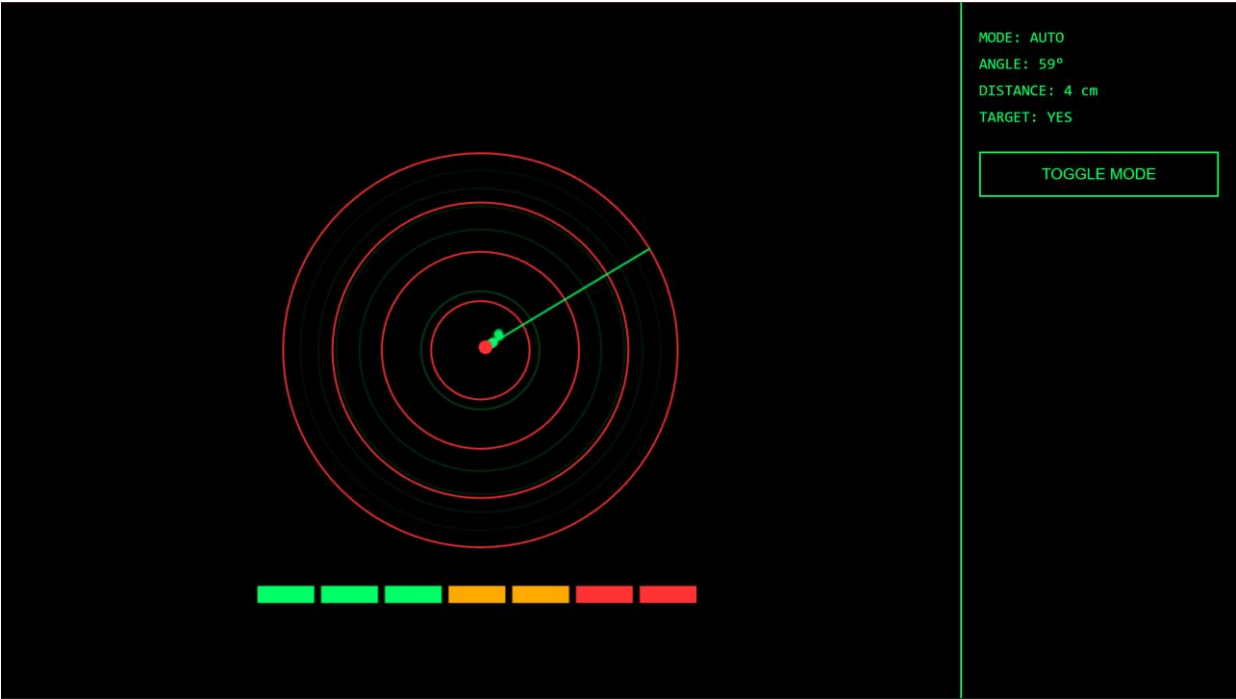
Beyond hardware control, I developed a live Wi-Fi radar dashboard hosted directly on the ESP32. The dashboard renders a dynamic green radar interface with animated sweep beams, wave pulses, target persistence effects, and a synchronized virtual LED ladder — creating a seamless bridge between embedded systems, control logic, signal filtering, and real-time web-based UI visualization.

This project reflects my approach to robotics: not just assembling components, but integrating sensing, actuation, feedback systems, and user interface design into a cohesive, scalable embedded platform.

Images:







Code:

```
#include <ESP32Servo.h>
#include <WiFi.h>
#include <WebServer.h>

/* ===== WIFI ===== */
const char* ssid = "ROBOT_RADAR";
const char* password = "12345678";
WebServer server(80);

/* ===== UI HTML ===== */
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta charset="UTF-8">
<title>Robot Radar</title>
<style>
body {
  margin: 0;
  background: black;
  color: #00ff66;
  font-family: monospace;
  display: flex;
  height: 100vh;
}
#radarArea {
  flex: 4;
  display: flex;
  justify-content: center;
  align-items: center;
}
#panel {
  flex: 1;
  padding: 20px;
  border-left: 2px solid #00ff66;
}
canvas { background: black; }
.label { margin: 10px 0; font-size: 18px; }
button {
  margin-top: 20px;
  width: 100%;
  padding: 14px;
  background: black;
  color: #00ff66;
  border: 2px solid #00ff66;
  font-size: 18px;
}
button:hover { background: #00ff66; color: black; }
</style>
</head>

<body>
<div id="radarArea">
  <!-- BIGGER CANVAS -->
  <canvas id="radar" width="640" height="640"></canvas>
</div>

<div id="panel">
  <div class="label">MODE: <span id="mode">---</span></div>
  <div class="label">ANGLE: <span id="angle">---</span>°</div>
  <div class="label">DISTANCE: <span id="distance">---</span>
  <div class="label">TARGET: <span id="locked">---</span></div>
  <button onclick="toggleMode()">TOGGLE MODE</button>
</div>
```



```

<script>
const canvas = document.getElementById("radar");
const ctx = canvas.getContext("2d");

const cx = canvas.width / 2;
const cy = canvas.height / 2;

// BIGGER RADAR
const radius = canvas.width / 2 - 90;

let angle = 0;
let distance = -1;
let mode = "AUTO";
let locked = false;

let pulses = [];
let echoes = [];
let lastPulseTime = 0;

const LED_COUNT = 7;

function fetchData() {
  fetch("/data")
    .then(r => r.json())
    .then(d => {
      angle = d.angle;
      distance = d.distance;
      mode = d.mode;
      locked = d.locked;

      document.getElementById("mode").innerText = mode;
      document.getElementById("angle").innerText = angle;
      document.getElementById("distance").innerText = distance;
      document.getElementById("locked").innerText = locked ? "YES" : "NO";

      if (locked && distance > 0 && distance < 100) {
        echoes.push({ a: angle, d: distance, alpha: 1.0 });
      }
    });
}

function updatePulses() {
  const now = Date.now();
  if (mode === "AUTO" && !locked && now - lastPulseTime > 600) {
    pulses.push({ r: 0, alpha: 0.35 });
    lastPulseTime = now;
  }
  pulses.forEach(p => { p.r += 3; p.alpha -= 0.004; });
  pulses = pulses.filter(p => p.alpha > 0 && p.r < radius);
}

/* ----- BIGGER LED BAR ----- */
function drawLEDBar() {
  const barX = 60;
  const barY = canvas.height - 45;
  const barWidth = canvas.width - 120;
  const barHeight = 20; // THICKER

  let lit = 0;
  if (distance > 0) {
    lit = Math.round((100 - distance) / (100 / LED_COUNT));
    lit = Math.min(LED_COUNT, Math.max(0, lit));
  }
}

```

```

for (let i = 0; i < LED_COUNT; i++) {
  let color = "#003300";
  if (i < lit) {
    if (i < LED_COUNT * 0.4) color = "#00ff66";
    else if (i < LED_COUNT * 0.7) color = "#ffaa00";
    else color = "#ff3333";
  }
  ctx.fillStyle = color;
  ctx.fillRect(
    barX + i * (barWidth / LED_COUNT),
    barY,
    barWidth / LED_COUNT - 8,
    barHeight
  );
}
}

function drawRadar() {
  ctx.clearRect(0,0,canvas.width,canvas.height);
  const flashRed = locked && Math.floor(Date.now()/300)%2===0;

  ctx.strokeStyle = flashRed ? "#ff3333" : "#00ff66";
  ctx.lineWidth = 2;

  ctx.beginPath();
  ctx.arc(cx, cy, radius, 0, Math.PI*2);
  ctx.stroke();

  for(let i=1;i<=4;i++){
    ctx.beginPath();
    ctx.arc(cx, cy, radius*i/4, 0, Math.PI*2);
    ctx.stroke();
  }

  pulses.forEach(p=>{
    ctx.strokeStyle=`rgba(0,255,102,${p.alpha})`;
    ctx.beginPath();
    ctx.arc(cx,cy,p.r,0,Math.PI*2);
    ctx.stroke();
  });

  const rad=(angle-90)*Math.PI/180;
  ctx.strokeStyle="#00ff66";
  ctx.beginPath();
  ctx.moveTo(cx,cy);
  ctx.lineTo(cx+radius*Math.cos(rad),cy+radius*Math.sin(rad));
  ctx.stroke();

  echoes.forEach(e=>{
    const er=(e.d/100)*radius;
    const ea=(e.a-90)*Math.PI/180;
    ctx.fillStyle=`rgba(0,255,102,${e.alpha})`;
    ctx.beginPath();
    ctx.arc(cx+er*Math.cos(ea),cy+er*Math.sin(ea),5,0,Math.PI*2);
    ctx.fill();
    e.alpha-=0.01;
  });
  echoes=echoes.filter(e=>e.alpha>0);

  if(distance>0&&distance<100){
    const r=(distance/100)*radius;
    ctx.fillStyle=flashRed?"#ff3333":"#00ff66";
    ctx.beginPath();
    ctx.arc(cx+r*Math.cos(rad),cy+r*Math.sin(rad),8,0,Math.PI*2);
    ctx.fill();
  }

  drawLEDBar();
}

function toggleMode(){
  fetch("/control?mode="+ (mode=="AUTO"? "MANUAL": "AUTO"));
}

```



```

setInterval(()=>{
    fetchData();
    updatePulses();
    drawRadar();
},100);
</script>
</body>
</html>
)rawliteral";

/* ===== HARDWARE CODE (UNCHANGED) ===== */

#define TRIG_PIN 5
#define ECHO_PIN 18
#define SERVO_PIN 13
int ledPins[]={16,17,19,21,22,23,25};
#define LED_COUNT (sizeof(ledPins)/sizeof(ledPins[0]))
#define JOY_X_PIN 34
#define JOY_BTN_PIN 14
#define BUZZER_PIN 4

#define MAX_DISTANCE 100
#define LOCK_DISTANCE 25
#define LOOP_INTERVAL 30
#define DEBOUNCE_TIME 250

enum Mode{AUTO,MANUAL};
Mode currentMode=AUTO;

bool targetLocked=false;
long lastDistance=-1;

Servo headServo;
float currentAngle=90,targetAngle=90,sweepStep=0.6;

unsigned long lastLoopTime=0,lastBtnTime=0,lastBeepTime=0;

long getDistanceCM(){
    digitalWrite(TRIG_PIN,LOW);delayMicroseconds(2);
    digitalWrite(TRIG_PIN,HIGH);delayMicroseconds(10);
    digitalWrite(TRIG_PIN,LOW);
    long d=pulseIn(ECHO_PIN,HIGH,25000);
    if(d==0)return -1;
    return d*0.034/2;
}

void updateProximityLEDs(long d){
    int n=map(d,MAX_DISTANCE,LOCK_DISTANCE,0,LED_COUNT+1);
    n=constrain(n,0,LED_COUNT);
    for(int i=0;i<LED_COUNT;i++) digitalWrite(ledPins[i],i<n);
}

void updateSound(long d){
    if(currentMode==MANUAL||d<=0||d>80){noTone(BUZZER_PIN);return;}
    if(d<=15){tone(BUZZER_PIN,2000);return;}
    int interval=map(d,80,15,1000,120);
    if(millis()-lastBeepTime>interval){
        tone(BUZZER_PIN,2000,50);
        lastBeepTime=millis();
    }
}

void handleRoot(){ server.send_P(200,"text/html",index_html); }

void handleData(){
    String j="{";
    j+="\"distance\":"; j+=lastDistance; j+=",";
    j+="\"angle\":"; j+=(int)currentAngle; j+=",";
    j+="\"mode\":\":"; j+=(currentMode==AUTO?"AUTO":"MANUAL"); j+="\"";
    j+="\"locked\":"; j+=(targetLocked?"true":"false");
    j+="}";
    server.send(200,"application/json",j);
}

```

```

void handleControl(){
    if(server.hasArg("mode")){
        if(server.arg("mode")== "AUTO")currentMode=AUTO;
        if(server.arg("mode")== "MANUAL")currentMode=MANUAL;
    }
    server.send(200,"text/plain","OK");
}

void setup(){
    Serial.begin(115200);
    pinMode(TRIG_PIN,OUTPUT); pinMode(ECHO_PIN,INPUT);
    for(int i=0;i<LED_COUNT;i++) pinMode(ledPins[i],OUTPUT);
    pinMode(JOY_BTN_PIN,INPUT_PULLUP); pinMode(BUZZER_PIN,OUTPUT);

    headServo.setPeriodHertz(50);
    headServo.attach(SERVO_PIN,500,2400);

    WiFi.softAP(ssid,password);
    server.on("/",handleRoot);
    server.on("/data",handleData);
    server.on("/control",handleControl);
    server.begin();
}

void loop(){
    server.handleClient();
    if(millis()-lastLoopTime<LOOP_INTERVAL)return;
    lastLoopTime=millis();

    if(digitalRead(JOY_BTN_PIN)==LOW&&millis()-lastBtnTime>DEBOUNCE_TIME){
        currentMode=currentMode==AUTO?MANUAL:AUTO;
        lastBtnTime=millis();
    }

    long d=getDistanceCM();
    lastDistance=d;
    updateProximityLEDs(d<0?MAX_DISTANCE:d);

    if(currentMode==AUTO){
        if(!targetLocked){
            targetAngle+=sweepStep;
            if(targetAngle>=180||targetAngle<=0)sweepStep=-sweepStep;
            if(d>0&&d<=LOCK_DISTANCE)targetLocked=true;
        }else{
            if(d<0||d>LOCK_DISTANCE+10)targetLocked=false;
        }
    }else{
        targetAngle=map(analogRead(JOY_X_PIN),0,4095,0,180);
        targetLocked=false;
    }

    currentAngle+=(targetAngle-currentAngle)*0.15;
    headServo.write(currentAngle);
    updateSound(d);
}

```

Project Development Methodology-Phase Wise Implementation:

The robotic scanning system was developed in a structured, phase-wise manner to ensure modular testing, system stability, and progressive feature integration.

Phase 1 – Servo Sweep Control (Mechanical Scanning Foundation)

The project began with implementing PWM-based servo control using the ESP32. The servo was programmed to sweep continuously between 0° and 180°, reversing direction at boundary limits.

Key Achievements:

- Stable PWM signal generation on ESP32
- Smooth incremental angle control
- Direction inversion logic at angular limits

Outcome:

A functional mechanical scanning head capable of continuously monitoring its surroundings.

Phase 2 – Ultrasonic Sensing Integration (Perception Layer)

The HC-SR04 ultrasonic sensor was integrated to measure object distance. The echo pulse duration was captured and converted into distance (cm). The servo was programmed to freeze when an object was detected within 20 cm.

Key Achievements:

- Time-of-flight distance measurement
- Real-time sensor-to-actuator interaction
- Basic object detection logic

Outcome:

The robot transitioned from passive scanning to active environment-aware behavior.

Phase 3 – Proximity LED Ladder (Visual Feedback System)

A graded LED ladder (green → yellow → orange → red) was implemented to visually represent object proximity. Distance values were mapped proportionally to the number of LEDs activated.

Key Achievements:

- Distance-to-LED mapping logic
- Real-time visual proximity indication
- Enhanced user interpretability

Outcome:

The robot now communicates spatial awareness visually, similar to automotive parking systems.

Phase 4 – Intelligent Audio Feedback (Sound Personality Layer)

A dynamic buzzer system was implemented with variable beep intervals based on distance.

Behavior Design:

- Slow beeping when object is far
- Faster beeping as distance decreases
- Continuous alarm tone at lock distance

This required non-blocking timing using `millis()` to ensure simultaneous servo motion and sound output.

Outcome:

The system gained realistic behavioral feedback resembling real-world parking sensors.

Phase 5 – Dual Mode Control & Wi-Fi Radar Dashboard (System Integration Layer)

The final phase integrated hardware control with a real-time web interface.

Features Added:

- **Auto Mode:** Autonomous scanning with validated object locking
- **Manual Mode:** Joystick-based servo control
- ESP32 Wi-Fi SoftAP server
- Real-time radar visualization including:
 - Sweep animation
 - Echo persistence
 - Distance display
 - Mode indicator
 - Virtual LED ladder synchronized with physical hardware

Outcome:

The project evolved into a complete embedded robotics platform combining sensing, actuation, feedback systems, control logic, and interactive visualization.

System Architecture:

The system is built around the **ESP32 microcontroller**, which acts as the central processing and control unit. It interfaces with sensing, actuation, feedback, and networking subsystems.

Block-Level Architecture

Input Layer

- HC-SR04 Ultrasonic Sensor → Distance Measurement
- Analog Joystick → Manual Servo Control
- Push Button → Mode Switching

Processing Layer

- ESP32 (32-bit dual-core MCU)
 - Distance computation
 - Lock validation logic
 - State machine (AUTO / MANUAL)
 - PWM generation for servo
 - LED proximity mapping

- Audio feedback timing
- Wi-Fi SoftAP server

Output Layer

- Micro Servo → Angular scanning (0°–180°)
- LED Ladder → Visual proximity indication
- Piezo Buzzer → Parking-sensor style audio alerts
- Web Dashboard → Real-time radar visualization

The ESP32 acts as both:

- Embedded controller
- Real-time web server

Embedded Web Dashboard Architecture:

The ESP32 runs a **Wi-Fi SoftAP server**, creating a local network.

Backend (ESP32)

- Hosts HTTP server
- Serves HTML dashboard
- Provides JSON endpoint /data
- Streams:
 - Current angle
 - Distance
 - Mode
 - Lock state

Frontend (Browser)

Built using:

- HTML
- CSS
- JavaScript Canvas API

Features:

- Animated radar sweep beam
- Echo persistence visualization

- Lock flash animation
- Virtual LED ladder synced with hardware
- Real-time updates via periodic fetch()

Detailed Control Logic-Decision Making Architecture:

1 System Level Decision Flow

Every (LOOP_INTERVAL), the machine executes one full decision cycle:

Step 1: Mode Check

- Read joystick button
- If pressed and debounce satisfied
- Toggle between AUTO and MANUAL

This defines the *control authority*.

Step 2: Distance Acquisition

The ultrasonic sensor is triggered:

- 10 μ s trigger pulse
- Measure echo pulse width
- Convert time → distance

Then raw signal validation occurs:

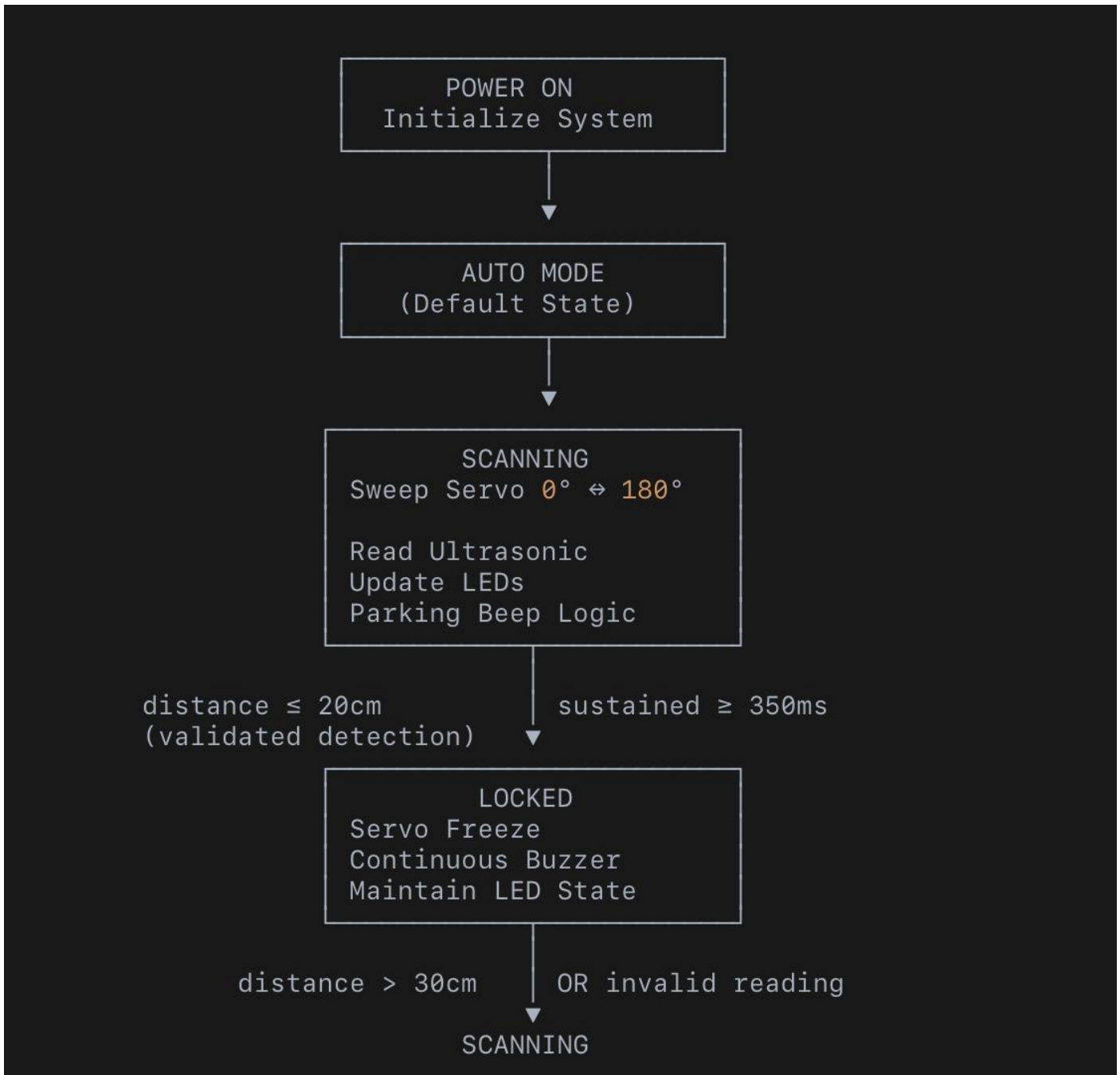
If:

- Pulse too small → reject
- Pulse too large → reject
- No echo → treat as invalid

Only validated distances enter the decision engine.

This prevents false mechanical reactions.

2 AUTO Mode Full Decision Logic



A. Object Detection Logic

Condition:

If $\text{distance} > 0$ AND $\text{distance} \leq \text{LOCK_DISTANCE}$ (20 cm)

The system does NOT immediately freeze.

Instead:

Lock Validation Process

1. If not already locked:
 - Start a lock timer
2. Object must remain inside range for LOCK_TIME_MS (~350 ms)
3. Only then:
 - targetLocked = true

This ensures:

- No single-sample noise
 - No servo freeze from spikes
 - No transient echo artifacts
-

B. Lock Maintenance Logic

Once locked:

If distance > UNLOCK_DISTANCE (25–30 cm)

OR distance invalid

Then:

- Unlock
- Reset lock timer
- Resume scanning

This hysteresis prevents oscillation between states.

C. Servo Control Logic (AUTO)

If NOT locked:

1. targetAngle += sweepStep
2. If targetAngle hits 0° or 180°:
 - Reverse direction

This produces continuous scanning.

If locked:

- targetAngle remains constant
 - Servo freezes at detected angle
-

D. Servo Motion Smoothing

Instead of jumping to target:

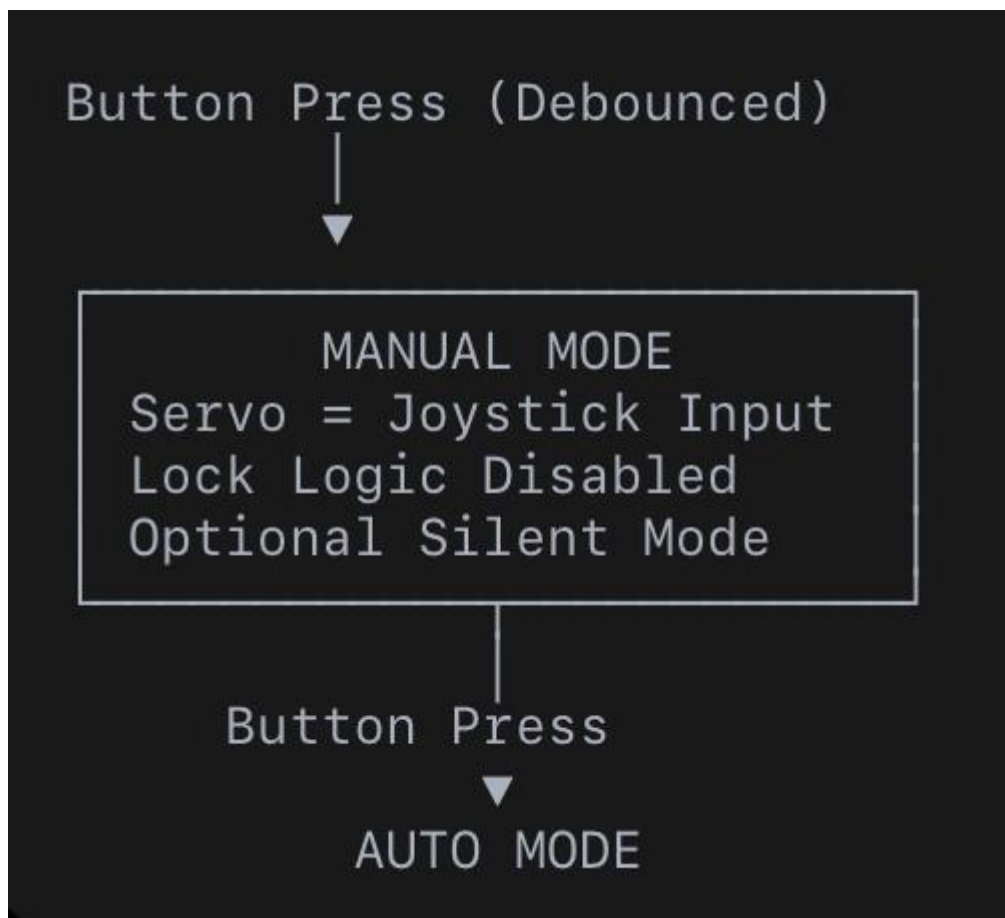
```
currentAngle += (targetAngle - currentAngle) * 0.15
```

This:

- Creates inertial motion
- Prevents jitter
- Reduces mechanical stress
- Mimics real radar behavior

3 MANUAL Mode – Control Override

MANUAL mode bypasses all autonomous logic.



Decision Structure:

1. Read analog joystick value (0–4095)
2. Map to 0–180°
3. Directly set targetAngle

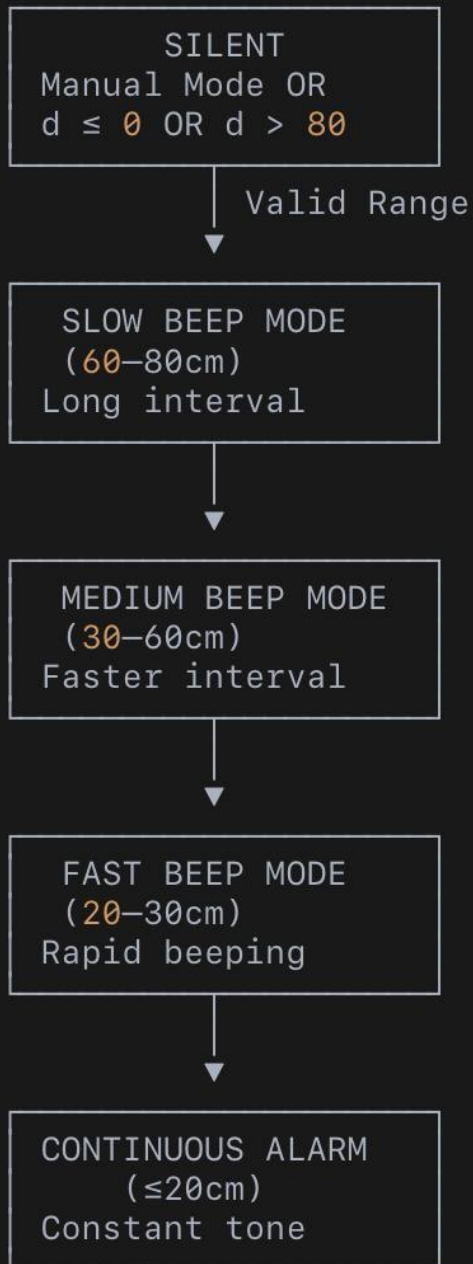
Then:

- Disable lock
- Disable lock timer

- Silence buzzer logic

Manual mode is a priority override state.

Buzzer Decision Engine



The buzzer runs its own logic layer independent of servo logic.

Silence Conditions:

- Manual mode
- Distance invalid
- Distance > 80 cm

Continuous Alarm Condition:

If distance $\leq$ LOCK_DISTANCE
→ Continuous 2000 Hz tone

Progressive Beeping Condition:

If LOCK_DISTANCE $<$ distance ≤ 80 cm

Beep interval:

`map(distance, 80, LOCK_DISTANCE, 1000ms, 150ms)`

Closer object → faster beeps

This mimics automotive parking sensor behavior.

5 LED Proximity Ladder Logic

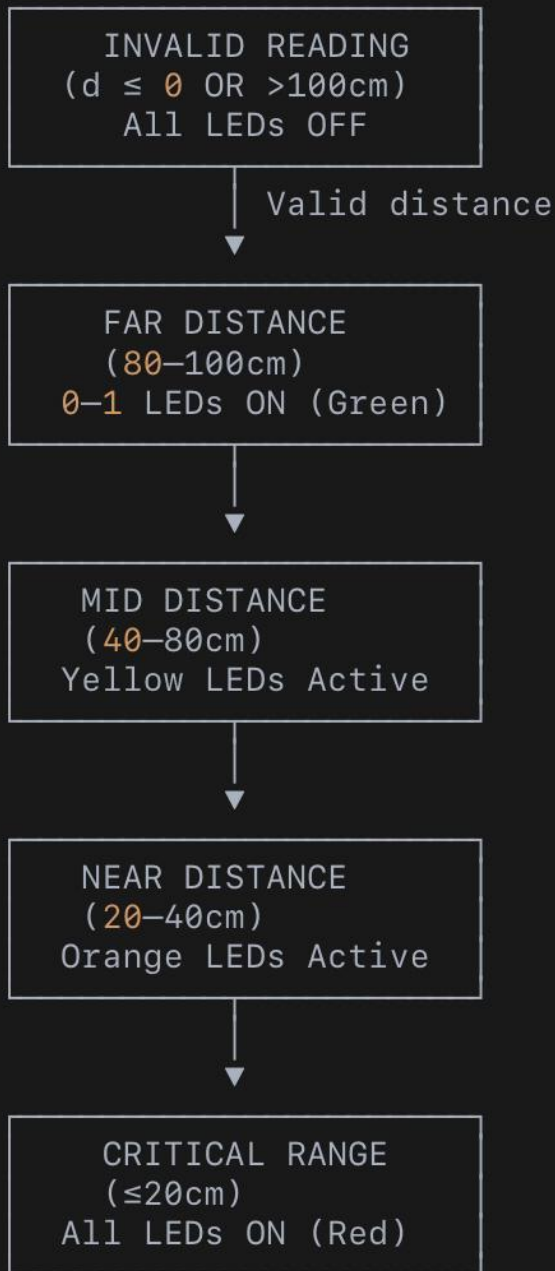
Distance is mapped to number of LEDs lit.

Mapping rule:

- Far → Few LEDs
- Close → More LEDs

This gives:

- Instant visual feedback
- Redundant sensory output
- Physical reinforcement of detection



6 Time Synchronization Layer

All decisions run inside a non-blocking timed loop:

```
if (millis() - lastLoopTime < LOOP_INTERVAL) return;
```

This ensures:

- Consistent update rate (~33 Hz)
- Stable servo timing
- Predictable behavior
- No blocking delays

7 Full Decision Hierarchy Summary

Each cycle:

1. Read button → decide mode
2. Measure distance → validate signal
3. Update LEDs
4. Run state machine:
 - AUTO or MANUAL branch
5. Update servo position (smoothed)
6. Run buzzer logic

The system is layered:

Signal Layer → Validation Layer → State Machine → Actuation Layer → Feedback Layer

Challenges Faced during Development

1. Ultrasonic Sensor Noise & False Locking

During early testing, transient distance spikes occasionally triggered premature servo locking, especially in simulation.

To solve this, I implemented signal validation using pulse rejection limits, hysteresis (20 cm lock / 25–30 cm release), and time-based confirmation to ensure stable object detection before freezing the servo.

2. Servo Freezing Outside Lock Region (Simulation-Specific)

In Wokwi simulation, the servo sometimes froze even when the measured distance was greater than the locking threshold — a behavior not observed in physical hardware.

3. Power Instability in Manual Mode

When controlling the servo aggressively using the joystick, the ESP32 occasionally reset due to sudden current spikes drawn by the servo motor.

This will be addressed by improving power routing and ensuring stable supply conditions to prevent voltage drops, in later versions

4. Mode Switching Debounce Issues

The joystick button initially caused unintended rapid toggling between Auto and Manual modes due to mechanical bounce.

A software debounce mechanism was implemented to ensure stable and reliable mode transitions.

5. Real-Time Web Dashboard Synchronization

Maintaining smooth synchronization between hardware states (angle, distance, lock status) and the live Wi-Fi radar dashboard required careful timing design.

A lightweight JSON data exchange system with controlled update intervals and client-side animation smoothing was implemented to ensure seamless real-time visualization.

My Learnings from this project:

1. Sensor Reliability is Harder Than It Looks

I learned that ultrasonic sensors produce noise, false echoes, and unstable readings — especially in simulation. Implementing validation logic, hysteresis, and time-based locking taught me how real-world embedded systems must handle imperfect signals.

2. Control Logic Requires Structured Thinking

Designing the servo freeze behavior forced me to think in terms of state machines rather than simple if statements. I understood how systems transition between states (Scanning → Locking → Locked → Release) based on validated conditions.

4. Power Management Matters in Robotics

I observed servo instability in manual mode due to power limitations, which taught me that hardware constraints directly affect control behavior. Embedded design is not just coding — it is electrical engineering too.

5. Embedded + Web Integration Expands Capability

Hosting a live Wi-Fi dashboard using SoftAP and WebServer showed me how embedded systems can interface with user-friendly web UIs. I learned how hardware state can be streamed as JSON and visualized in real time.

6. Simulation ≠ Real Hardware

The unexpected servo freezing in Wokwi helped me understand that simulators may behave differently from physical hardware.

7. System Thinking Over Component Thinking

This project shifted my mindset from “making components work” to “designing an integrated system”, combining sensing, actuation, signal filtering, UI, and reliability into a cohesive platform.
